

### 1.4.1 Multiplication \*

Consider the following code:

```
int x = 2;
double y = 3.889, sum = 0.000;
sum = y * x;
```

The integer will be temporarily converted to a double and two doubles will be multiplied. Afterwards, the original integer remains unchanged.

Rules for temporary conversions:

First priority: If either of the operands is of type double, then the other operand is converted to double for the calculation.

Second priority: If either of the operands is of type float, then the other operand is converted to float for the calculation.

Third Priority: If any of the operands is of type long, then the other operand is converted to long for the calculation.

Note: these conversions are automatic, because none of them result in loss of accuracy.

#### Static casts

When you convert a double to float, then information is inevitably lost. You can accomplish this using a cast:

```
int x = 2, sum = 0;
double y = 3.889;
sum = (int)y * x;
{ sum is now equal to 6 }
```

Here, a value of just 3 will be used for y. If you want to round y, you a method from class Math:

```
sum = (int)Math.round(y) * x;
```

### 1.4.2 Division /

Division can lead to unexpected results. For example, if both operands are integers, then the result of the division is also an integer. Any fractional part of the division is discarded. As a result,  $17/3 = 5$ .

### 1.4.3 Modulus %

In contrast to the division operator, modulus returns the remainder of any division. The modulus operator can only be used when both operands are integers. Thus,  $17 \% 3 = 2$ .